

Master IARFID

Reconocimiento de Escritura (RES)

Practical session: Training

Moisés Pastor and PRHLT Group



Departament de Sistemes Informàtics i Computació
Universitat Politècnica de València



February 13, 2024

Description

- ▶ Deep Neural Networks are the state-of-the-art technology for handwritten text recognition.
- ▶ We will make a complete HTR system based on neural networks.
- ▶ First, we will train an optical model based on Deep Neural Networks with Connectionist Temporal Classification (CTC).
- ▶ Secondly, we will decode the test set with and without language model.

Installing software

```
#Download the tool
git clone https://github.com/moisesPastor/HTRnn.git

# Create a new conda environment
python3.10 -m venv res_env

# or
conda create -n res_env python=3.9
conda init
conda activate res_env

# or
virtualenv res_env

# Activate the environment
source res_env/bin/activate

# Installing dependencies
pip install -r HTRnn/requirements.txt

# Give execution permission
chmod +x HTRnn/train.py HTRnn/test.py \
HTRnn/get_confMats*
```

Installing software

```
# Install the required package "ctcdecode"
# Check at: https://github.com/parlance/ctcdecode.git

git clone --recursive https://github.com/parlance/ctcdecode.git
cd ctcdecode

# Modify the setup.py to allow ngram order up to 10

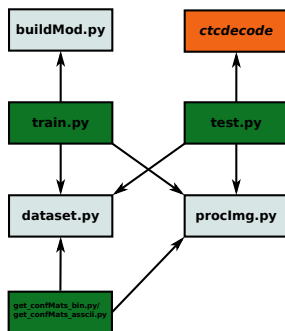
sed -r "/-DKENLM_MAX_ORDER=/\
s/-DKENLM_MAX_ORDER=[0-9]+/-DKENLM_MAX_ORDER=10/" \
    setup.py > aux

mv aux setup.py
pip install . #python setup.py install

cd ../..
```

Overview of the HTRnn System

Basically our HTR system architecture is composed of 7 modules. The dependency relationships among these modules are shown in the Fig.1.



Overview of the HTRnn System: `proclmg.py`

- ▶ This module defines the processing step to apply to input images.
- ▶ Previously established transformations are applied to each line image before it were fed into the neural network model.
- ▶ There are three transformations whose application is mandatory:
 1. Scale line images to a fixed-height.
 2. Invert pixel values of images. Usually, gray scale images represent black pixels (handwritten strokes) as 0 and white (background) as 255, whereas the opposite is more reasonable in this case.
 3. Convert images to tensors in **PyTorch**.

Overview of the HTRnn System: **proclmg.py**

```
# See details in HTRnn/proclmg.py
...
def get_tranform(fixedHeight=64, dataAug=False):
...

# List of transformations for Data Augmentation
daTr = [ ... ] if dataAug else []

# List of mandatory transformations
nTr = [ ... ]

return transforms.Compose( dtaTr + nTr )
```

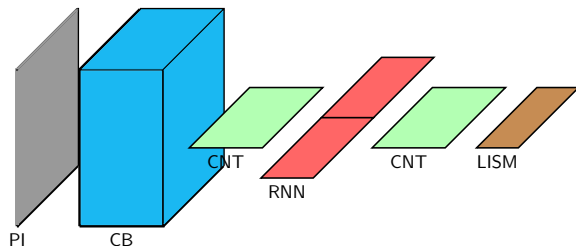
Overview of the HTRnn System: **buildMod.py**

- ▶ This module defines the network topology.
- ▶ The *forward* function defines the data pipeline through different layers to finally reach the last one with C outputs (the different trained character classes).
- ▶ In this case, the outputs are produced through a *log-softmax* layer.

```
# See details in HTRnn/model.py
...
class HTRModel(nn.Module):
def __init__(self, num_classes, line_height=32):
    super(HTRModel, self).__init__()
    ...

def forward(self, x):
    ...
    return F.log_softmax(x, dim=2)
```


Overview of the HTRnn System: **buildMod.py**



Reference Parameters:

- PI:** Processed Image, **Output-Size:**40 (of fixed height)
- CB:** **Cnn-Knl/Strd:**3×3/1×1, **Feat-Mps:**64, **Act-F:**Relu, **MxPool:**2×1
- CNT:** Concat layer, after the CB concatenates all (64) Feat-Maps from all pixels of an individual column into a single vector.
- RNN:** **Rnn-type:**BLSTM, **Units:**100+100
- LISM:** Linear+Softmax layer

Figure: The topology consists of one *Convolutional Block* (CB), one *Concat layer* (CNT) followed by one *Recurrent Neural Network* layer (RNN) and one full connected *Linear+Softmax* layer (LISM).

Overview of the HTRnn System: **dataset.py**

- ▶ **PyTorch** provides two classes: *Dataset* and *DataLoader* to facilitate the management of samples for model training and evaluation.
 - *Dataset* is conceived to store samples and their corresponding labels.
 - *DataLoader* wraps an iterable around *Dataset* to enable easy access to the samples.
- ▶ The *HTRDataset* is a specialized class derived from *Dataset* to manage our set of samples.

```
class HTRDataset(Dataset):
    # A custom Dataset class must implement __init__ function
    def __init__(self, root_dir, spaceChar, transform=None, **kargs):
        self.root_dir = root_dir
        ...
    # A custom Dataset class must implement __len__ function
    def __len__(self):
        ...
    # A custom Dataset class must implement __getitem__ function
    def __getitem__(self, idx):
        ...
```

Overview of the HTRnn System: **train.py**

This module implements the training pipeline process. Basically the following things are carried out:

- ▶ set up transforms to apply to each image (including setting a fixed-height value for line images).
- ▶ instantiate our `htr_dataset` through the custom implemented class *HTRDataset*.
- ▶ set up a mini-batch-size through instantiating the *DataLoader* class.
- ▶ instantiate model object through the *HTRModel* class.
- ▶ define the loss function (*criterion*) used for computing network convergence (*CTCLoss*).
- ▶ define an *optimizer* method used to change weights and learning rate to reduce the loss in the training process.

Overview of the HTRnn System: **train.py**

```
# See details in HTRnn/train.py
...
def train(ocr_dataset, epochs=20, ...):
    ...
    #img_transforms = procimg.get_tranform(args.max_height)
    ...
    #htr_dataset = HTRDataset(dataPth, img_transforms, ...)
    ...
    train_loader = torch.utils.data.DataLoader(htr_dataset, ...)
    ...
    model = HTRModel(num_classes, line_height)
    ...
    criterion = torch.nn.CTCLoss(blank=BLANK, zero_infinity=True)
    ...
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
    ...
```

Overview of the HTRnn System: **train.py**

```
# Epoch loop
for epoch in range(epochs):
    ...
    # Mini-batch loop
    for x, y in train_loader:
        ...
        outputs = model(x)
        ...
        loss = criterion(outputs, y, ...)
        ...
        loss.backward()
        ...
        optimizer.step()
        ...
```

Overview of the HTRnn System: **test.py**

- ▶ This module incorporates the “Connectionist Temporal Classification Beam Search Decoding” (*CTCBeamDecoder*), which is imported by the python package `ctcdecode`. The **C++** code in this package has been borrowed from the source-code of the Paddles’ DeepSpeech.
- ▶ *CTCBeamDecoder* can perform the plain traditional CTC-decoding and the one with language model re-scoring in case a n -gram (in ARPA format) were provided. For the case of language model re-scoring, the library OpenFST is used to.

Overview of the HTRnn System: **test.py**

This module has the mini-batch loop where the forward pass is performed and its output fed into the CTC-based decoder. Finally, the outputted tensors are decoded into text lines to then be compared with their references for evaluation.

```
def test(model, htr_dataset, ...):
    ...
    test_loader = torch.utils.data.DataLoader(ocr_dataset, ...)
    ...
    decoder = CTCBeamDecoder(ocr_dataset.get_charVoc(), lmFile, ...)
    ...
    # Main Mini-batch loop
    for (x, y) in test_loader:
        ...
        outputs = model.forward(x)
        ...
        output, scores, ... = decoder.decode(outputs.data, ...)
        ...
        # Decode outputted tensors into text lines
    # Compute WER/CER
    ...
```

Training the optical model

1. Preparing transcriptions

```
mkdir data/text
for f in data/txt/*; do
    gawk -v fileName=$(basename $f .txt) '{
        printf("%s_r0_l%02i ", fileName, NR-1);
        print
    }' $f;
done | sed -e "s/\s+/ /g" \
    -e "s/^\s+//g" \
    -e "s/[\$\\t]/ /g" \
    -e "s/_/ /g" > /tmp/transcriptions.txt
```


Training the optical model

2. Getting the units map

```
mkdir -p data/lists

gawk '{
    $0=tolower($0)
    $1=""
    n=split($0,symbols,"")
    for(j=1;j<=n;++j)
        if (symbols[j] != " ")
            print symbols[j],symbols[j]
}' /tmp/transcriptions.txt | sort -u > data/lists/unitsMap.lst
```

3. Simplify the set of optical units. Edit "data/lists/unitsMap.lst" and change, or delete the symbols on the second column

- Units in the first column will be replaced by the ones in the second when we normalize the ground truth.
- If there is not a symbol on the second column, it will be deleted on the ground truth.
- Just leave ASCII characters on the second column or empty.

Training the optical model

4. Getting the ground truth at optical units level

```
sed -e "s/\~{\S*}//g" -e "s/[[][\~]*[]//g" \
    /tmp/transcriptions.txt | \
gawk -v MAP=data/lists/unitsMap.lst 'BEGIN{
    while ((getline < MAP) > 0)
        if (NF == 2) DICT[$1] = $2
    DICT[" "] = " ";
}{
    printf ("%s ",$1)
    for(w=2; w<=NF; w++) {
        N=split(tolower($w),CHAR,"")
        for(i=1; i<=N; i++)
            if (CHAR[i] in DICT)
                printf("%s",DICT[CHAR[i]])
        printf (" ")
    }
    printf("\n")
}' |tr -s [[:space:]] > data/text/transcriptions.txt
```

Prepare corpus

5. Getting partitions: training, validation and test files list;

- training files list

```
for file in `cut -d" " -f1 data/text/transcriptions.txt`; do  
  [ -e data/Corpus_clean_lines/${file}.jpg ] && echo $file;  
done | head -15000 > data/lists/train.lst
```

- validation files lists

```
for file in `cut -d" " -f1 data/text/transcriptions.txt`; do  
  [ -e data/Corpus_clean_lines/${file}.jpg ] && echo $file;  
done | head -16000 | tail -1000 > data/lists/val.lst
```

- test files list

```
for file in `cut -d" " -f1 data/text/transcriptions.txt`; do  
  [ -e data/Corpus_clean_lines/${file}.jpg ] && echo $file;  
done | tail -5346 > data/lists/test.lst
```

Prepare corpus

6. Prepare the transcriptions files

```
awk '{
    FILE=$1; $1=""
    print FILE
    print $0 > "data/Corpus_clean_lines/"FILE".txt"
}' data/text/transcriptions.txt
```

7. Create the corpus partions for train, validation and test

```
mkdir data/data_train data/data_test data/data_val

for dataset in {train,test,val}; do
    for file in `cat data/lists/${dataset}.lst`; do
        echo $file;
        ln -s ${PWD}/data/Corpus_clean_lines/$file.{txt,jpg} \
            data/data_${dataset}
    done
done
```

Running a model training process

8. Getting the optical symbols list to train adding some special ones.

```
sort -u -k 2 data/lists/unitsMap.lst | gawk 'BEGIN{
    print "<eps>";
    print "<ctc>";
    print "~";
    }{if (NF==2 && !($2 in AP)) {AP[$2]=1; print $2}}' \
> data/lists/symbols_train.lst
```

9. Run a HTR model training process on samples in data/data_train

```
mkdir -p models/Optical

HTRnn/train.py \
--models-file data/lists/symbols_train.lst \
--data-augm --epochs 120 \
--space-symbol "~" --batch-size 15 \
data/data_train data/data_val/ models/Optical/rodrigo.pth
```

Running a model training process

```
usage: train.py [-h] --models-file MODELS_FILE [--data-augm]
               [--fixed-height FIXED_HEIGHT] [--epochs EPOCHS]
               [--early-stop EARLY_STOP] [--batch-size BATCH_SIZE]
               [--space-symbol SPACE_SYMBOL] [--gpu GPU [GPU ...]]
               dataset_train dataset_val model_name
```

Run a model training process using the given dataset.

positional arguments:

dataset_train train dataset location

dataset_val validation dataset location

model_name save model with this file name

options:

-h, --help show this help message and exit

--models-file MODELS_FILE file with models name (default: None)

--data-augm enable data augmentation (default: False)

--fixed-height FIXED_HEIGHT fixed image height (default: 64)

--epochs EPOCHS number of epochs (default: 20)

--early-stop N stops if no improvement after N epochs (default: 10)

--batch-size BATCH_SIZE image batch-size (default: 24)

--space-symbol SPACE_SYMBOL image batch-size (default: ~)

--gpu GPU [GPU ...] used gpu (default: 0)

Testing Model and Evaluation

1. Run a plain recognition and evaluation process on samples in data/data_test. Change BEST_MODEL for the latest model.

```
HTRnn/test.py --batch_size 25 --verbosity  
models/Optical/BEST_MODEL data/data_test/
```

```
#####  
# Model's params: 1728479  
    CER: 4.39%  
    WER: 22.81%  
#####
```

2. To include a language model in the decoding process, we train an 8-gram model on the transcripts belonging to the training samples. To this end, the following steps are done:

Testing Model and Evaluation

```
# Before LM training, we separate word characters and replace  
# the space characters (that separate words) by "~"
```

```
mkdir -p models/LM
```

```
cat data/data_train/*.txt data/data_val/*.txt | \  
awk '{  
    gsub(" ", "~", $0);  
    n=split($0, CHARS, "");  
    for (i=2; i<=n; i++)  
        printf("%s ", CHARS[i]); printf("\n")  
}' | ngram-count -text - -order 8 -wbdisc1 \\  
    -kndisc1 -interpolate -lm models/LM/8-gram.arpa
```


Testing Model and Evaluation

1. The produced 8-gram language model is applied for re-scoring, along with different values of *gsf* and *wip* for improving performance:

```
# Set up "grammar scale factor" (gsf) and
# "word insertion penalty" (wip)
# Change BEST\_MODEL for the latest model.

HTRnn/test.py --verbosity --batch_size 24 \
  --lm-model models/LM/8-gram.bin --gsf 0.8 \
  --wip 3.0 models/Optical/BEST_MODEL data/data_test

#####
# Model's params: 1728479
# CER: 1.03%
# WER: 2.51%
#####
```

Testing Model and Evaluation

```
usage: test.py [-h] [--verbosity] [--gpu GPU [GPU ...]]
               [--lm-model LM_MODEL] [--beam-width BEAM_WIDTH]
               [--gsf GSF] [--wip WIP] [--batch_size BATCH_SIZE]
               model dataset
```

Testing a HTR model.

positional arguments:

model PyTorch model file

dataset dataset path

options:

```
-h, --help show this help message and exit
--verbosity increase output verbosity (default: False)
--gpu GPU [GPU ...] used gpu (default: [-1, 1])
--lm-model LM_MODEL language model file (Arpa/Kenlm format)
                        (default: None)
--beam-width BEAM_WIDTH this sets up how broad the beam search is
                        (default: 200)
--gsf GSF Grammar scale factor (default: 0.0)
--wip WIP Word insertion penalty (default: 0.0)
--batch_size BATCH_SIZE image batch-size (default: 24)
```